PrivacyGNN: Complete Technical Deep Dive

Every major concept, file, method, dependency, output, and limitation

Prepared for Krish Sharma

May 11, 2026

Abstract

This document is a deep technical guide to the **PrivacyGNN** project. It explains the research problem, graph-learning background, membership inference attacks, model architectures, defenses, experiment grid, code modules, output files, visualization scripts, paper-generation tools, dependencies, testing status, and known limitations. It is written so you can learn the project from first principles and then map each concept directly to the codebase.

Contents

1 Executive Summary

PrivacyGNN studies whether graph neural networks leak membership information: can an attacker infer whether a node was part of the training set by looking at the model's prediction probabilities? The codebase is a reproducible experiment framework for node-classification privacy experiments.

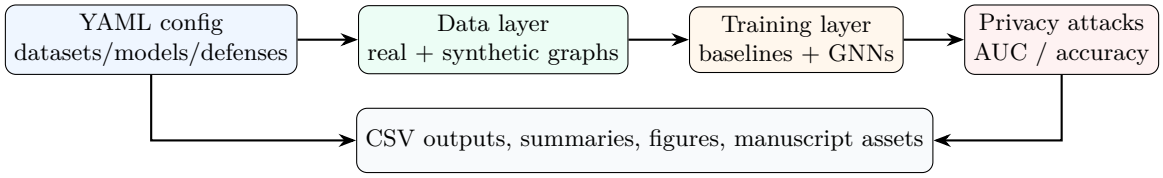
The central workflow is:

1. Load or generate a graph dataset.
2. Split nodes into train and test members.
3. Train a model: feature-only baseline or GNN.
4. Optionally apply a lightweight defense.
5. Use prediction probabilities to run membership inference attacks.
6. Save utility, privacy, calibration, and graph-structure metrics.
7. Aggregate results, test defense effects, bootstrap uncertainty, and generate figures.

Important reality check for this checkout:

- Current results contain **560 rows**: 8 datasets, 4 models, 6 defenses for GNNs, 5 seeds.
- `lira_attack.py` is a placeholder returning chance-level scores.
- `ogb_loader.py` and `graph_minibatch.py` are stubs; large-graph OGB/Reddit/minibatch/DP-SGD paths are not active in this snapshot.
- There is no dedicated unit-test directory or pytest suite discovered. Validation is currently through reproducible runs, CSV outputs, plotting scripts, and statistical summaries.

2 Project at a Glance



The project is mostly **Python**. It also uses **YAML** for configuration, **Bash** for orchestration, **LaTeX** and **ReportLab** for PDFs, **Markdown** for analysis notes, and **CSV** files for experiment outputs.

3 Repository Structure

The active repository root is `privacy-gnn/`. The table below explains every important source artifact.

File / folder	Role in the project
`README.md`	Human-facing overview, setup commands, project structure, research questions, and notes about newer extensions versus paper configuration.
`requirements.txt`	Python dependency list: PyTorch, PyTorch Geometric, OGB, Opacus, NumPy, sklearn, pandas, scipy, matplotlib, seaborn, ReportLab, PyYAML, and plotting helpers.
`config.py`	Central configuration loader. Reads YAML, provides defaults, converts defense dictionaries into tuples, builds the experiment grid, and creates output directories.

File / folder	Role in the project
experiment_config.yaml	Main large config. Uses 10 seeds, includes LiRA placeholder and DP-SGD/large graph settings, though large-graph execution is stubbed here.
experiment_config_paper.yaml	Paper-aligned grid: 5 seeds, Cora, Citeseer, six synthetics, confidence/threshold/shadow attacks, no LiRA or DP-SGD.
data.py	Dataset loading and graph utilities: Cora/Citeseer loading, synthetic graph generation, train/test resplitting, homophily, density, and random edge dropping.
models.py	Two GNN architectures: two-layer GCN and two-layer GraphSAGE. Baselines are not here; they are sklearn models in experiment.py.
training.py	Full-batch GNN training loop with DropEdge, label smoothing, early stopping, and edge sparsification support.
attacks.py	Core membership inference attacks and calibration: feature extraction, confidence attack, threshold attack, shadow attack, and ECE.
experiment.py	The single-run engine. Loads one dataset/model/defense/seed combination, trains target and shadow models, runs attacks, and returns a flat metrics row.
stats_utils.py	Statistical utilities: paired t-tests defense-vs-none, LiRA t-test wrapper, bootstrap confidence intervals over seeds.
run_final.py	Main runner. Builds full experiment grid, iterates over runs, writes all results, summary, significance, LiRA significance, and bootstrap CSVs.
generate_figures.py	Publication-style six-figure generator from results/all_results.csv; includes citation fallbacks and synthetic plots.
generate_basic_figures.py	Simpler four-figure synthetic visualization script from summary.csv.
generate_final_visuals.py	Polished visual suite for final presentation/poster, outputting to ../Final Visuals.
generate_poster_visuals.py	Larger poster-ready visual suite: pipeline, heatmaps, privacy-utility plots, defense rankings, significance maps, calibration plots, and summary table.
fix_figures.py	Patch script for improved versions of Figure 2 and Figure 4, using adjustText for labels.
ogb_loader.py	Stub. MINIBATCH_DATASETS is empty and load_large_benchmark raises NotImplementedError.
graph_minibatch.py	Stub. Minibatch training, minibatch inference, and DP-SGD minibatch training all raise NotImplementedError.
lira_attack.py	Placeholder. lira_gaussian_auc returns (0.5, 0.5).
run_everything_final.sh	Bash script that runs run_final.py and then generate_figures.py.
RESULTS_ANALYSIS.md	Human-written interpretation of synthetic results and research questions.
paper/ieee_privacy_gnn.tex	Main IEEE-style manuscript source.
paper/build_manuscript.py	ReportLab script that creates a static workshop-style manuscript.pdf. Some claims in this script should be checked against current code/results before reuse.
paper/build_project_slides.py	PowerPoint generation script using python-pptx. Produces a designed project overview deck.
paper/literature_review.md	Literature notes supporting the paper framing.
results/*.csv	Experiment outputs: per-run rows, summaries, significance tests, bootstrap CIs, and partial checkpoint.
data/	Downloaded PyG datasets and raw Planetoid files.
docs/	Learning PDFs generated for you, including this technical guide.

4 Languages, File Types, and Tooling

Type	Where	Why it is used
Python	`*.py`	Core experiments, model definitions, training, attacks, statistics, figures, manuscript/deck builders.
YAML	`experiment_config*.yaml`	Reproducible experiment definitions without editing source code.
Bash	`run_everything_final.sh`	One-command orchestration for full run plus figures.
CSV	`results/*.csv`	Durable tabular output, easy to analyze with pandas/spreadsheets.
LaTeX	`paper/*.tex`, `docs/*.tex`	Academic paper and guide PDFs.
Markdown	`README.md`, `RESULTS_ANALYSIS.md`, `paper/*.md`	Human-readable documentation and interpretation.
PowerPoint	`paper/build_project_slides.py` output	Presentation deck generation.
PNG/PDF figures	`figures/`, `../Final Visuals/`	Publication/poster/presentation visuals.

5 Dependencies and Libraries

Library	Purpose in this project
`torch`	Tensor computation, neural network modules, optimization, softmax/logits, GNN training loops.
`torch-geometric`	Graph data object `Data`, Planetoid dataset loading, GCNConv, SAGEConv.
`ogb`	Listed for future large benchmark support, but current loader is stubbed.
`opacus`	Listed for DP-SGD privacy accounting, but current DP path is stubbed.
`numpy`	Random seeds, synthetic data generation, feature extraction, bootstrap sampling.
`scikit-learn`	Logistic regression, MLP baseline, attack classifiers, metrics.
`pandas`	CSV reading/writing, groupby aggregation, pivot tables for figures/statistics.
`scipy`	Paired t-tests through `scipy.stats`.
`matplotlib`	All generated figures.
`seaborn`	Heatmaps and visual style in publication figures.
`PyYAML`	Parse experiment configuration files.
`reportlab`	Programmatic PDF manuscript creation in `paper/build_manuscript.py`.
`adjustText`	Improved label placement in `fix_figures.py`.
`tqdm`	Listed dependency, not central in the current code paths.

6 Research Concepts You Need to Understand

6.1 Graph Node Classification

The basic supervised task is node classification. Each node has:

- feature vector x_i,
- label y_i,
- graph neighbors through `edge_index`,
- membership in train or test masks.

The model predicts a class distribution $p(y \mid x_i, G)$. For baselines, G is ignored. For GNNs, G is essential because features are mixed through message passing.

6.2 Membership Inference

A membership inference attack asks:

$$\text{Given model output for node } i, \quad \text{was } i \text{ in the training set?}$$

In this project:

- train-mask nodes are treated as **members**,
- test-mask nodes are treated as **non-members**,
- attack AUC near 0.5 means random guessing,
- attack AUC above 0.5 means membership leakage.

6.3 Graph Homophily

Homophily is computed in `data.py` as:

$$\text{homophily} = \frac{\#\{(u,v) \in E : y_u = y_v\}}{\#E}.$$

High homophily means connected nodes usually share labels. Low homophily means edges often connect different labels.

6.4 Graph Density

Density is computed as:

$$\text{density} = \frac{\text{undirected edges}}{n(n-1)/2}.$$

Sparse graphs have fewer neighbor messages; dense graphs have more smoothing and more structural signal.

7 Data Layer in Detail

7.1 `data.py`: Citation Datasets

`load_citation(name, data_dir)` uses `torch_geometric.datasets.Planetoid` to load Cora or Citeseer. It returns:

(`data`, `num_classes`, `num_features`)

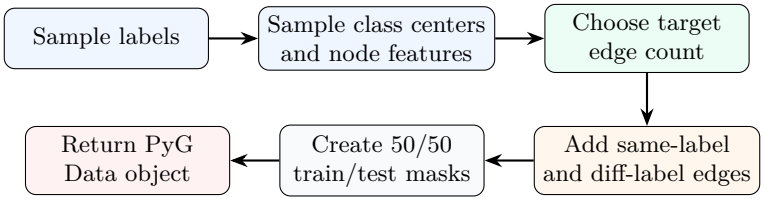
where `data` contains `x`, `edge_index`, `y`, and masks. If the default Planetoid URL fails, the function temporarily swaps `Planetoid.url` to a mirror URL.

7.2 `data.py`: Synthetic Graph Generator

`make_synthetic(n=400, nf=50, nc=5, homo="high", dens="medium", seed=42)` is one of the project's most important methods. It creates controlled graph regimes:

- $n = 400$ nodes by default.
- $nf = 50$ features per node.
- $nc = 5$ classes.
- Labels are random class IDs.
- Class centers are Gaussian vectors.
- Node features are class center plus Gaussian noise.
- Density controls how many edges are sampled.

- Homophily controls same-label versus different-label edge fraction.
- Train/test split is 50/50.



7.3 Resplitting and Reproducibility

`resplit(data, seed)` clones a graph and creates a fresh 50/50 train/test split. This means different seeds change which nodes are members and non-members. This is critical for paired comparisons across defenses.

7.4 Edge Dropping

`drop_edges(edge_index, rate)` randomly keeps each edge with probability $1 - \text{rate}$. It is used by:

- DropEdge during training,
- edge sparsification before training,
- explicit edge preprocessing in `experiment.py` for minibatch/DP paths.

8 Model Layer

8.1 Feature-only Baselines

The baselines are defined inside `experiment.py`, not `models.py`:

- `LogisticRegression(max_iter=1000)`
- `MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=200)`

Both consume `data.x.numpy()` and ignore `edge_index`. They are important controls: they tell you how much privacy leakage comes from features and classifier confidence alone.

8.2 GCN

`models.py` defines:

$$\text{GCNConv}(d_{\text{in}}, 64) \to \text{ReLU} \to \text{Dropout}(0.5) \to \text{GCNConv}(64, C)$$

GCN uses graph convolution, where each layer aggregates information from neighbors using normalized adjacency. It is usually strong when homophily is high.

8.3 GraphSAGE

GraphSAGE follows the same two-layer layout, but uses `SAGEConv`. It aggregates neighbor features differently and is often more robust in inductive or noisy graph settings.

8.4 Why Two GNNs Matter

GCN and GraphSAGE are both message-passing models, but they do not behave identically. The result analysis notes that GCN can leak more in low-homophily sparse settings, while GraphSAGE often stays closer to random attack AUC.

9 Training Layer and Defenses

`training.py` is the full-batch GNN trainer. It takes a model, a PyG `Data` object, and training options.

9.1 Base Optimization

The base training loop:

1. Move data/model to device.
2. Create Adam optimizer.
3. Optionally sparsify edges once.
4. For each epoch:
   (a) optionally DropEdge for this epoch,
   (b) forward pass,
   (c) compute loss on `train_mask`,
   (d) backpropagate,
   (e) optimizer step,
   (f) optionally early-stop.

9.2 Defense Methods

Defense	Where applied	Technical behavior
`none`	Training/prediction baseline	No modification. Used as the reference for significance tests.
`dropedge`	Training-time	Drops random edges each epoch. In config, default rate is 0.3.
`label_smoothing`	Loss function	Replaces one-hot targets with softened class distributions. Config alpha is 0.1.
`early_stopping`	Training control	Tracks loss improvement and restores best state if patience expires. Config patience is 15.
`confidence_masking`	Post-prediction	Keeps top-k class probabilities and renormalizes. Config top_k is 2.
`edge_sparsification`	Pre-training graph	Removes a fraction of edges once before training. Config rate is 0.2.
`dp_sgd`	Stub path	Config exists, but actual minibatch DP function is not implemented in this checkout.

9.3 Important Implementation Detail

Confidence masking is not part of model training. In `experiment.py`, it is applied after probabilities are produced:

$$p_i \leftarrow \text{keep top-}k(p_i), \quad p_i \leftarrow p_i / \sum_j p_{ij}.$$

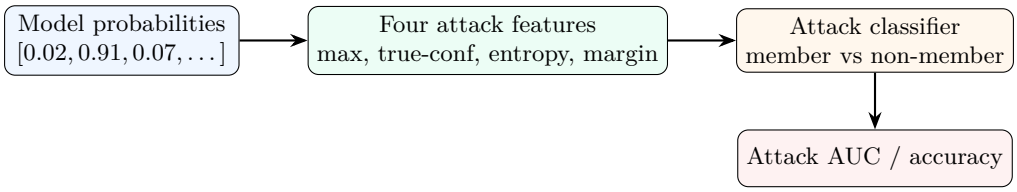
That means it is an inference-time output defense.

10 Attack Layer

10.1 Attack Feature Map

`attacks.py` converts each probability vector into a four-dimensional attack feature:

$$\phi(p_i, y_i) = \begin{bmatrix} \max_j p_{ij} \\ p_{i,y_i} \\ -\sum_j p_{ij} \log(p_{ij}) \\ -(1 - \max_j p_{ij}) \log(\max_j p_{ij}) \end{bmatrix}.$$



10.2 Confidence Attack

`confidence_attack` stacks member features and non-member features, labels them 1/0, shuffles, trains logistic regression on half, and evaluates on half. It returns:

$$(\text{confidence AUC},\ \text{confidence accuracy},\ \text{threshold AUC},\ \text{threshold accuracy})$$

This attack is simple but powerful because models often assign higher confidence to training samples.

10.3 Threshold Attack

The threshold attack uses only true-label confidence p_{i,y_i}. It computes ROC AUC from the raw confidence scores and an accuracy from median-threshold predictions.

10.4 Shadow Attack

`shadow_attack` trains an attacker on outputs from a shadow model:

1. Train a shadow model on a separate split/synthetic draw.
2. Label shadow train nodes as members and shadow test nodes as non-members.
3. Train attack logistic regression on shadow features.
4. Evaluate attack model on target-model member/non-member features.

10.5 LiRA Placeholder

`lira_attack.py` currently contains:

```
return 0.5, 0.5
```

So LiRA values in results are chance-level placeholders unless that file is replaced with a real implementation. Any paper claim based on LiRA should not be made from this checkout as-is.

10.6 Expected Calibration Error

`calibration_error` bins samples by max confidence and sums:

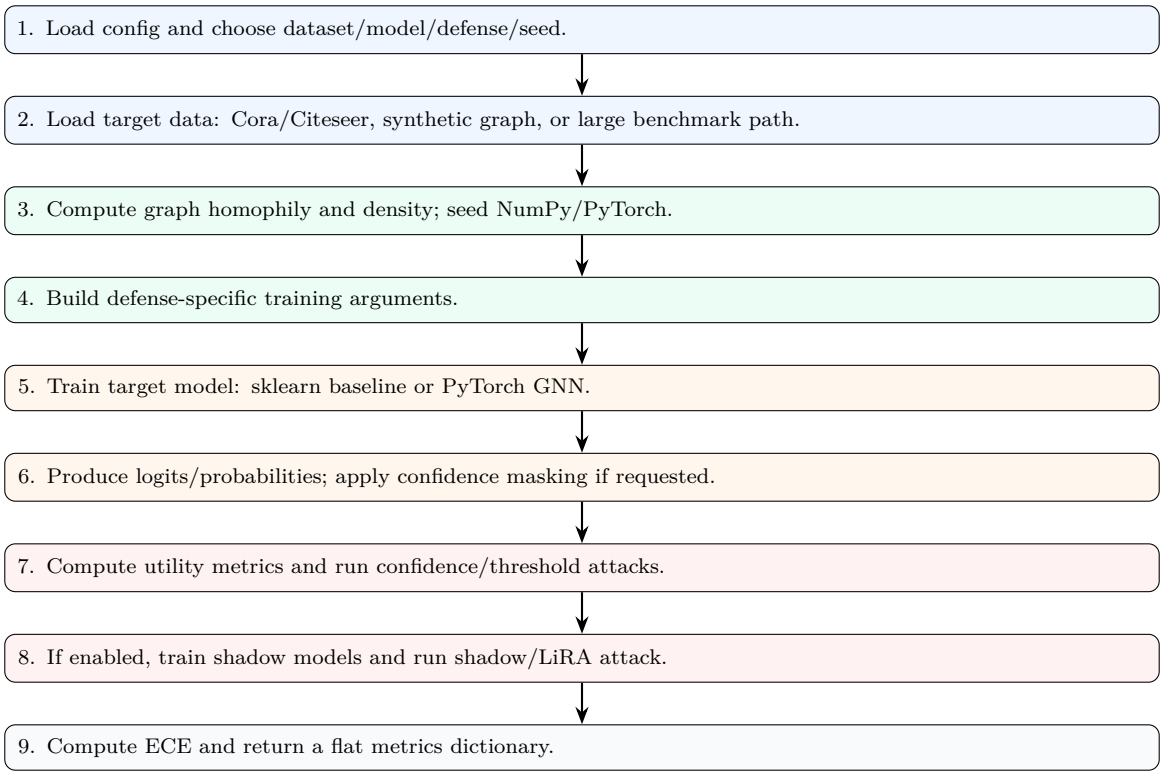
$$\text{ECE} = \sum_b \frac{|B_b|}{n} \left|\text{acc}(B_b) - \text{conf}(B_b)\right|.$$

ECE is not an attack. It measures whether confidence values are honest. Poor calibration can correlate with higher leakage.

11 The Single-Run Engine: `experiment.py`

`experiment.py` is the heart of the project. Its main public function is `run_one`. It produces exactly one row in `all_results.csv`.

11.1 High-Level Call Flow



11.2 Dataset Dispatch

`_load_target_data` uses dataset name rules:

- `Cora`, `Citeseer`: load Planetoid then resplit.
- `synthetic_*_*`: parse the name into homophily and density.
- names in `MINIBATCH_DATASETS`: large-graph path, currently unreachable because the set is empty.

11.3 Model Dispatch

- `GCN` and `GraphSAGE`: PyTorch model + `training.py`.
- `LogReg`: sklearn logistic regression.
- `MLP`: sklearn MLP classifier.

11.4 Returned Metrics

The row contains:

- identifiers: dataset, model, defense, params, seed,
- graph stats: homophily, density,
- utility: test accuracy, macro F1, test AUROC,
- attack metrics: confidence, threshold, shadow, LiRA,

- calibration: ECE,
- privacy budget: DP epsilon if DP-SGD ran.

12 Configuration System

12.1 Default Config

`config.py` contains built-in defaults, but normally it reads YAML. The environment variable `PRIVACYGNN_CONFIG` can override the config path:

```
PRIVACYGNN_CONFIG=experiment_config_paper.yaml python run_final.py
```

12.2 Grid Construction

`get_experiment_list` loops:

$$\text{datasets} \times \text{models} \times \text{defenses} \times \text{seeds}.$$

It skips invalid combinations:

- LogReg and MLP only run `none`.
- DP-SGD only applies to GCN/GraphSAGE.
- DP-SGD can be restricted to certain datasets.

12.3 Current Result Grid

The current `results/all_results.csv` contains:

- 560 rows and 20 columns,
- 8 datasets: Cora, Citeseer, and six synthetic regimes,
- 4 models: LogReg, MLP, GCN, GraphSAGE,
- 6 defenses in the result table,
- 5 seeds: 42, 123, 456, 789, 1024.

13 Main Runner and Output Files

13.1 `run_final.py`

`run_final.py` performs the production run:

1. load config,
2. ensure directories,
3. build experiment list,
4. loop over all combinations,
5. call `run_one`,
6. periodically checkpoint partial results,
7. write full results,
8. aggregate summary,
9. run significance tests,
10. run bootstrap summary.

13.2 Output CSVs

File	Shape	Meaning
`results/all_results.csv`	560 x 20	One row per complete run. This is the raw experimental table.
`results/summary.csv`	112 x 16	Grouped by dataset/model/defense, with means/stds.
`results/significance.csv`	80 x 7	Paired t-tests for confidence attack AUC, defense vs no defense.
`results/significance_lira.csv`	80 x 7	Same structure for LiRA AUC, but LiRA is placeholder here.
`results/summary_bootstrap.csv`	448 x 9	Bootstrap confidence intervals over seeds for metrics.
`results/all_results.partial.csv`	varies	Checkpoint written every 50 completed rows during long runs.

14 Statistics Layer

14.1 Paired t-tests

`stats_utils.py` compares each defense against `none`, paired by seed within the same dataset/model group. This is important because paired comparisons reduce noise from random splits.

$$d_s = \mathrm{AUC}_{\mathrm{defense},s} - \mathrm{AUC}_{\mathrm{none},s}$$

Then SciPy's paired t-test evaluates whether the mean difference is distinguishable from zero.

14.2 Bootstrap Confidence Intervals

The bootstrap function resamples seeds with replacement inside each dataset/model/defense group. It estimates uncertainty for metric means and writes percentile intervals:

$$[\mathrm{quantile}_{\alpha/2},\ \mathrm{quantile}_{1-\alpha/2}].$$

This is separate from p-values.

14.3 How to Interpret Statistics Carefully

The project uses only five seeds in the paper config/current results. That is useful but not huge. Treat p-values as exploratory evidence, not final proof. Bootstrap CIs are better for communicating uncertainty visually.

15 Visualization Scripts

15.1 `generate_figures.py`: Publication Figures

This script reads `all_results.csv` and produces six paper-oriented figures:

1. attack AUC vs model on Cora/Citeseer,
2. privacy-utility tradeoff,
3. leakage vs homophily,
4. defense effectiveness,
5. leakage vs density,
6. comprehensive heatmap.

It includes checks for missing Cora/Citeseer rows and falls back for Figure 2 if Cora GNN results are absent.

15.2 generate_basic_figures.py

This script reads `summary.csv` and focuses on synthetic summaries:

- grouped bar chart of attack AUC by model,
- GCN-only leakage by setting,
- GCN defenses on low-homophily medium setting,
- heatmap of model by setting.

15.3 generate_final_visuals.py

This script creates polished presentation graphics in `../Final Visuals`. It emphasizes:

- worst-case low-homophily sparse setting,
- GCN vs GraphSAGE trajectories,
- defense comparison,
- privacy-utility point cloud,
- attack-AUC matrix.

15.4 generate_poster_visuals.py

This is the most extensive visual suite. It creates 11 poster-ready outputs:

- study pipeline,
- leakage heatmap,
- privacy-utility landscape,
- defense delta heatmap,
- defense ranking,
- attack family comparison,
- homophily/density effects,
- calibration vs leakage,
- significance heatmap,
- regime small multiples,
- summary table.

15.5 fix_figures.py

This script regenerates improved versions of Figure 2 and Figure 4. It uses `adjustText` to reduce label overlap. It writes PNG and PDF versions.

16 Paper, Poster, and Slide Assets

16.1 paper/ieee_privacy_gnn.tex

This is the main IEEE-style manuscript source. Use it for academic writing, but keep it synchronized with current code and result files.

16.2 paper/build_manuscript.py

This script uses ReportLab to create a static PDF manuscript. It includes narrative text, tables, and references directly in Python. Because it is hand-written, it can drift from the live code. For example, code defaults in the actual pipeline use 50 epochs and 50/50 splits for many paths, while manuscript text may describe other protocol settings. Always verify claims against `experiment_config_paper.yaml`, `run_final.py`, and `all_results.csv`.

16.3 paper/build_project_slides.py

This script uses `python-pptx` to create a clean slide deck. It defines a small design system (colors, fonts, footer) and adds slides explaining the study, datasets, models, attacks, defenses, takeaways, and limitations.

17 How the Methods Differ

Method family	Specific methods	Key difference
Models	LogReg, MLP, GCN, GraphSAGE	Baselines ignore graph edges; GNNs use neighborhood aggregation.
Graph regimes	High/low homophily, sparse/medium/dense	Tests whether structure affects privacy leakage.
Defenses	DropEdge, smoothing, early stop, masking, sparsification	Some change training, some change graph, one changes output probabilities.
Attacks	Confidence, threshold, shadow, LiRA placeholder	Confidence/shadow learn an attack classifier; threshold is direct; LiRA is not implemented.
Statistics	t-tests, bootstrap CIs	t-tests compare defense vs none; bootstrap estimates uncertainty over seeds.
Visuals	Basic, paper, final, poster	Same data, different presentation goals and levels of polish.

18 Testing and Validation Status

18.1 What Exists

There is no dedicated automated unit-test suite in the current repository. No `tests/` directory, `test_*.py`, or pytest configuration was found in the project survey.

The project instead validates behavior through:

- reproducible config-driven execution,
- deterministic seeds,
- CSV outputs for every run,
- aggregation and paired comparisons,
- bootstrap uncertainty,
- figure generation from saved results,
- manual inspection through `RESULTS_ANALYSIS.md`.

18.2 Recommended Tests to Add

If you want engineering-grade confidence, add:

- unit tests for `make_synthetic`: node count, feature shape, mask sizes, density range, homophily trend;
- unit tests for `extract_features`: output shape and known probability cases;

- unit tests for `confidence_attack`: random synthetic probabilities give AUC near 0.5;
- smoke test for `run_one` on one tiny synthetic graph;
- config-grid test ensuring LogReg/MLP skip graph-only defenses;
- regression test ensuring `summary.csv` columns are stable;
- figure-generation smoke test using a small fake CSV.

19 Known Limitations and Technical Debt

Limitation	Technical meaning
LiRA placeholder	`lira_attack.py` returns 0.5 metrics. It is not a real LiRA implementation.
Large graphs stubbed	`ogb_loader.py` has an empty dataset set; `graph_minibatch.py` raises `NotImplementedError`.
DP-SGD path inactive	Config includes DP-SGD fields, but the implementation depends on stubbed minibatch DP training.
No automated tests	The project is reproducible but not unit-tested.
Some generated paper text may drift	ReportLab manuscript script is static text and should be checked against live result files.
Attack assumptions	Attacks use true labels in feature extraction. This is common in some MIA evaluations but should be stated clearly.
Only node-level classification	Link-level, graph-level, subgraph-level, and inductive privacy are not fully studied.

20 How to Learn the Codebase Efficiently

1. Start with `experiment_config_paper.yaml`; understand the grid.
2. Read `run_final.py`; understand how rows are created and aggregated.
3. Read `run_one` in `experiment.py`; this is the core control flow.
4. Read `data.py`; understand synthetic graph generation and masks.
5. Read `models.py` and `training.py`; understand GCN/GraphSAGE and defenses.
6. Read `attacks.py`; understand the four attack features and attack AUC.
7. Open `results/all_results.csv`; identify one row and trace how it was produced.
8. Open `summary.csv`; connect raw rows to means/stds.
9. Read `RESULTS_ANALYSIS.md`; compare interpretations against the CSVs.
10. Study the figure scripts last; they are presentation layers over saved results.

21 Command Cheat Sheet

```
# Install dependencies
cd privacy-gnn
pip install -r requirements.txt

# Run default configured grid
python run_final.py

# Run paper-aligned grid
PRIVACYGNN_CONFIG=experiment_config_paper.yaml python run_final.py
```

```
# Generate publication figures
python generate_figures.py

# Generate basic synthetic figures
python generate_basic_figures.py

# Generate final presentation visuals
python generate_final_visuals.py

# Generate poster visual suite
python generate_poster_visuals.py

# Build manuscript from ReportLab script
python paper/build_manuscript.py

# Build slide deck
python paper/build_project_slides.py
```

22 Final Mental Model

If you remember only one thing, remember this:

> **PrivacyGNN is a controlled experiment engine. The config defines a grid. `run_final.py` executes the grid. `experiment.py` creates one metrics row. `attacks.py` turns prediction confidence into membership-inference scores. The results CSVs are the source of truth for all figures and claims.**

The intellectual contribution is not merely "training a GNN." It is the systematic comparison of models, graph structures, defenses, and attacks under reproducible seeds, with graph structure measured and controlled.